

Technical Documentation

Credit Risk Modeling - Deployment, Inference and Retraining

This document describes the current technical implementation, engineering decisions, local operation, and items that still require evolution before a regulated production deployment.

Version 1.0 | Operational reference document
Repository: credit-risk-modeling

Contents

Contents	2
1. Objective and scope	3
2. Data and source	3
3. Notebook findings	3
4. Model and prediction contract	4
5. Deployment architecture	5
6. Configuration and dependency management	5
7. Container operations	5
8. Batch inference	6
9. Retraining and promotion	7
10. Monitoring and Power BI	8
11. Security, governance, and limitations	8
12. Delivery checklist	8

1. Objective and scope

This project estimates credit-default probability for loan applications in a batch-scoring setting. It covers relational data preparation, exploratory analysis, feature engineering and selection, training, model registration, and an operational foundation for inference, retraining, and monitoring.

The engineering objective is reproducibility and traceability: each prediction should be linked to the received batch, the model alias requested, the exact MLflow version resolved, and the decision threshold used.

Current scope: inference and retraining are manual Airflow DAGs. The infrastructure and monitoring schema are available; the inference DAG publishes the scored file and still needs to be connected to PostgreSQL prediction and monitoring fact writes.

2. Data and source

The dataset is derived from the Home Credit Default Risk competition published on Kaggle. Its licence, terms of use, and source attribution must be respected in any redistribution: <https://www.kaggle.com/c/home-credit-default-risk>.

The central table is application_train.csv, with one current application per SK_ID_CURR and the TARGET label. The other sources represent bureau history, previous applications, POS/cash balances, credit-card balances, and instalments. One-to-many sources are aggregated before joining to preserve one row per application.

File	Key / grain	Use
application_train.csv	SK_ID_CURR; current application	Base table and TARGET source
bureau.csv + bureau_balance.csv	External credit and monthly history	Aggregate by bureau, then customer
previous_application.csv	Previous application	Aggregate by customer
POS_CASH_balance.csv	Monthly POS/cash balance	Aggregate by customer
credit_card_balance.csv	Monthly card balance	Aggregate by customer
installments_payments.csv	Instalment payment	Aggregate by customer

Notebook 00 writes data/raw/conjunto_completo.csv; training, validation, and test splits are stored under data/interim/. Raw and interim data must not be committed to Git.

3. Notebook findings

The exploratory analysis covered 307,511 applications and found an 8.07% default rate. This class imbalance makes accuracy unsuitable as the primary measure; PR-AUC, recall, KS, and class weighting received more emphasis.

The analysis found meaningful missingness in housing and external-score variables: BASEMENTAREA_MEDI had 58.52% missingness and EXT_SOURCE_1 had 56.38%. Missingness indicators should be assessed together with imputation.

Stratified splits preserved the event rate: 8.0729% in training, 8.0728% in validation, and 8.0728% in test. Associations by contract type, gender, and outlier status were treated as descriptive monitoring signals, not causal decision rules.

Final test metric	Result
Precision	0.2569
Recall	0.4661

Final test metric	Result
F1-score	0.3313
PR-AUC	0.2789
ROC-AUC	0.7807
KS	0.4193
Threshold	0.6500

These are the registered final test results for the champion model. The test set was kept outside training, tuning, and threshold selection.

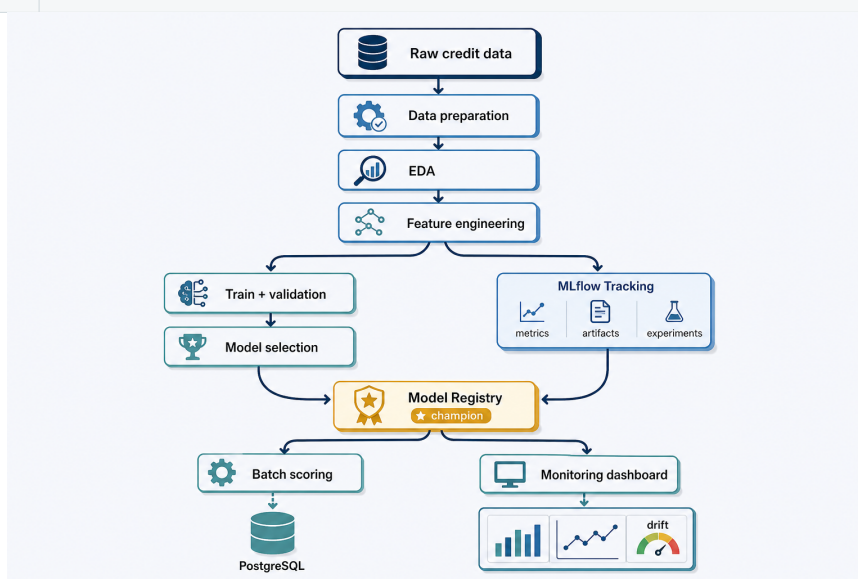
4. Model and prediction contract

The selected model is a binary LightGBM GBDT. The manual configuration was retained because Optuna tuning did not generalise better on validation. Hyperparameters are versioned in `src/config/model_config.yml`.

The pipeline uses `StandardScaler` for numeric columns and `OrdinalEncoder` for categorical columns. Unknown categories receive -1, allowing scoring of batches with new categories without encoder failure. Class weighting is computed from the training target.

Model selection uses PR-AUC in cross-validation. The threshold is selected on validation by maximising F1-score. Test data is reserved for final assessment and champion-versus-challenger comparison during retraining.

Contract	Definition
Input	67 selected features. The module performs feature engineering for raw data; when the full contract is already present, it only orders the columns.
Output	<code>probability_default</code> (double) and <code>default_prediction</code> (binary integer).
Registry	MLflow Model Registry: <code>credit-risk-model</code> ; operational alias: <code>champion</code> .
Traceability	Signature, input example, output example, threshold, metrics, and interpretability artefacts are stored in MLflow.



Project machine learning workflow.

5. Deployment architecture

Deployment is local and containerised through Docker Compose. PostgreSQL logically separates MLflow, Airflow, and monitoring databases; MLflow stores metadata in PostgreSQL and artefacts in a Docker volume; Airflow executes DAGs and accesses code and data through mounted volumes.

Service	Responsibility	Port / persistence
postgres	Database for MLflow, Airflow, and monitoring	Internal 5432; postgres_data volume
mlflow	Tracking, artefacts, and Model Registry	http://localhost:5000; artifacts_data volume
database-bootstrap	Creates airflow and monitoring databases and applies an idempotent migration	Runs and exits
airflow-init	Migrates Airflow metadata and creates the admin user	Runs and exits
airflow-webserver	UI for triggering and observing DAGs	http://localhost:8080
airflow-scheduler	Schedules and executes tasks	Internal service

The postgres_data, artifacts_data, and airflow_logs volumes survive a normal stop. Therefore, MLflow artefacts do not appear as ordinary files in the repository folder.

6. Configuration and dependency management

pyproject.toml declares direct dependencies and the supported Python range. uv.lock pins the complete resolution, including transitive dependencies; it must be committed and never manually edited. Run uv sync after changing the manifest or moving to a new machine.

Model compatibility is part of the operational contract. The current champion was serialised with Python 3.11.7, scikit-learn 1.9.0, LightGBM 4.4.0, Pandas 2.1.4, and Cloudpickle 2.2.1. The project and Airflow image were aligned with that runtime family. A new champion should only be promoted after validation in the inference runtime.

File	Role
.python-version	Requests Python 3.11 for compatible tools.
pyproject.toml	Dependency manifest and pytest/uv configuration.
uv.lock	Resolved, reproducible dependency graph.
docker/Dockerfile.airflow	Airflow 2.11.2 image with Python 3.11 and champion runtime.
docker/requirements-airflow.txt	Operational image dependencies pinned for safe model loading.
docker-compose.yml	Topology, volumes, ports, health checks, and service variables.

The .env file remains local and is ignored by Git. At minimum it must define POSTGRES_PASSWORD, AIRFLOW_FERNET_KEY, AIRFLOW_WEBSERVER_SECRET_KEY, AIRFLOW_ADMIN_PASSWORD, and MONITORING_KEY_SALT. It may also define database names and the administrator username. Passwords should be URL-safe because they are used in PostgreSQL URIs.

7. Container operations

Prerequisites: Docker Desktop, Docker Compose, and uv. On Windows, open the cloned repository in VS Code and use PowerShell from the project root.

Main commands:

```
uv sync
docker-compose up -d --build
docker-compose ps
docker-compose logs -f airflow-scheduler
docker-compose down
```

Use `docker-compose down -v` only when you deliberately want to delete persisted databases, artefacts, and logs.

After startup, MLflow is available at `http://localhost:5000` and Airflow at `http://localhost:8080`. Airflow credentials are defined in `.env` and must never be included in the repository or documentation.

Recommended operational checks: `uv lock --check`, `docker-compose exec -T airflow-scheduler python -m pip check`, `docker-compose exec -T airflow-scheduler airflow dags list-import-errors`, and `health-check status` in `docker-compose ps`.

8. Batch inference

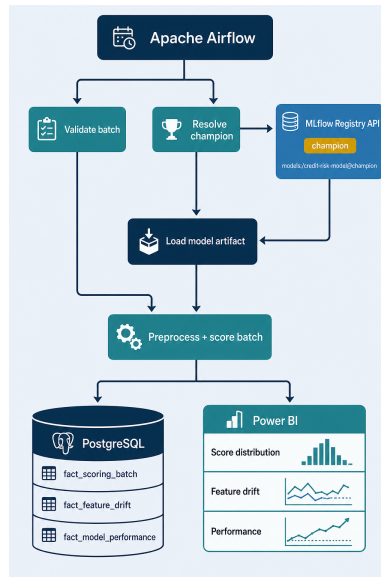
The `credit_risk_batch_inference` DAG is manual (`schedule=None`) and contains three tasks: `load_data`, `predict`, and `save_predictions`. The manual choice is intentional while the operational batch source is still being defined.

A run must receive paths visible inside the container. Since the local `data/` folder is mounted at `/opt/airflow/data`, a host file at `data/batches/input.parquet` must be referenced as `/opt/airflow/data/batches/input.parquet`.

Example Airflow run configuration:

```
{
  "input_path": "/opt/airflow/data/batches/input.parquet",
  "output_path": "/opt/airflow/data/scored/scored_2026_08.parquet"
}
```

Step	Behaviour
load_data	Validates file existence and CSV/Parquet extension and creates a <code>batch_id</code> .
predict	Loads <code>models:/credit-risk-model@champion</code> , resolves the concrete version, applies the feature contract, and appends the two outputs.
save_predictions	Writes first to <code>.staging</code> and publishes through an atomic move. Existing outputs are never overwritten.



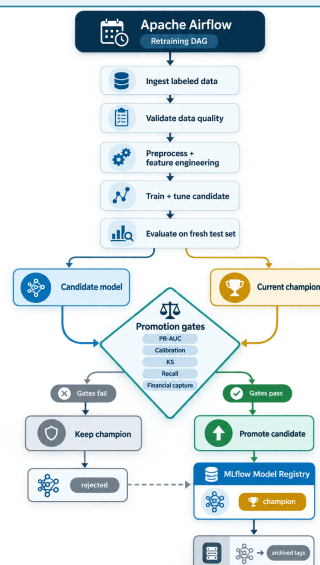
Planned batch inference and monitoring architecture.

9. Retraining and promotion

The `credit_risk_retraining` DAG is also manual and requires a labelled file containing TARGET. It validates the input, creates stratified train, validation, and test splits, trains the challenger, selects a validation threshold, and compares challenger and champion on the same test partition kept outside fitting.

The implemented promotion rule is strict: challenger PR-AUC must be higher than champion PR-AUC. The challenger is always registered, receives the challenger alias and validation tags. If approved, the champion alias moves to it and the former version receives `lifecycle_status=archived`. If rejected, the champion remains unchanged.

Important: for a production operation, replace the current random split with temporal windows and an out-of-time holdout. Before promotion, add calibration, KS, minimum recall, financial capture, data-quality, and fairness guardrails.



Challenger retraining and promotion architecture.

10. Monitoring and Power BI

The database/migrations/001_monitoring_star_schema.sql migration creates the monitoring schema in the monitoring database. The structure follows a star schema for Power BI; the calendar dimension belongs in the Power BI semantic model.

Dimensions: dim_model, dim_feature, dim_segment, dim_data_reference, and dim_validation_rule. Facts: fact_scoring_batch, fact_prediction, fact_training_observation, fact_training_feature_profile, fact_feature_monitoring, fact_prediction_monitoring, fact_data_quality_result, fact_directional_expectation_result, and fact_model_performance.

The training baseline links a model version to its features, target, and statistical profile. Application identifiers are anonymised with MONITORING_KEY_SALT before persistence. The future prediction load should record model version, batch, probability, threshold, decision, and observed target when available.

Area	Metrics and validations
Data quality	Volume, duplicates, required columns, type, range, regex, and allowed values.
Feature drift	PSI, missingness change, numeric quantiles, categorical distribution, and new categories.
Prediction drift	Score-distribution PSI, mean probability, and threshold-crossing rate.
Performance	PR-AUC, ROC-AUC, KS, precision, recall, F1, Brier, calibration, and captured financial exposure.
Directional expectation	Checks whether a feature-risk direction follows a business-approved hypothesis; it does not establish causality.

11. Security, governance, and limitations

Secrets remain in .env and must never appear in commits, notebooks, screenshots, or documentation. In a shared deployment, use a secrets manager and least-privilege database accounts.

The champion alias is mutable; every scoring event should persist the resolved model version and run_id, not only the alias. Registered MLflow models and artefacts should be considered immutable after registration.

The historical pathlib warning when loading the current artefact is a non-blocking inconsistency in legacy MLflow metadata. It is not equivalent to scikit-learn deserialisation warnings, which were removed by aligning the runtime. The registered artefact must not be manually edited.

Before real credit use, additional privacy, authorisation, audit, risk-approved explainability, independent validation, and data-retention controls are required.

12. Delivery checklist

1. Run `uv sync` and `uv lock --check`.
2. Populate .env without exposing secrets.
3. Start `docker-compose up -d --build`.
4. Confirm MLflow, Airflow, and PostgreSQL health.
5. Confirm that the champion alias exists in MLflow.
6. Test manual inference with a new input and output path.
7. Inspect the model schema, signature, and artefacts.
8. For retraining, use labelled data and review the comparison result before any regulated-environment promotion.